

# File permissions in Linux

## Project description

As a security professional working alongside our research team, my mission was to ensure that our file system wasn't a "leaky bucket." We had a situation where the actual file permissions didn't align with our official authorization levels. I stepped in to bridge that gap, ensuring that sensitive project data was only accessible to those who truly needed it.

## Check file and directory details

The following code demonstrates how I used Linux commands to determine the existing permissions set for a specific directory in the file system.

```
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w---- 1 researcher2 research_team   46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

The first line of the screenshot displays the command I entered, and the other lines display the output. The code lists all contents of the projects directory. I used the `ls` command with the `-la` option to display a detailed listing of the file contents that also returned hidden files. The output of my command indicates that there is one directory named `drafts`, one hidden file named `.project_x.txt`, and five other project files. The 10-character string in the first column represents the permissions set on each file or directory.

## Describe the permissions string

The 10-character string can be deconstructed to determine who is authorized to access the file and their specific permissions. The characters and what they represent are as follows:

- **1st character:** This character is either a `d` or hyphen (`-`) and indicates the file type. If it's a `d`, it's a directory. If it's a hyphen (`-`), it's a regular file.
- **2nd-4th characters:** These characters indicate the read (`r`), write (`w`), and execute (`x`) permissions for the user. When one of these characters is a hyphen (`-`) instead, it indicates that this permission is not granted to the user.

- **5th-7th characters:** These characters indicate the read (r), write (w), and execute (x) permissions for the group. When one of these characters is a hyphen (-) instead, it indicates that this permission is not granted for the group.
- **8th-10th characters:** These characters indicate the read (r), write (w), and execute (x) permissions for other. This owner type consists of all other users on the system apart from the user and the group. When one of these characters is a hyphen (-) instead, that indicates that this permission is not granted for other.

For example, the file permissions for `project_t.txt` are `-rw-rw-r--`. Since the first character is a hyphen (-), this indicates that `project_t.txt` is a file, not a directory. The second, fifth, and eighth characters are all r, which indicates that user, group, and other all have read permissions. The third and sixth characters are w, which indicates that only the user and group have write permissions. No one has execute permissions for `project_t.txt`.

## Change file permissions

The goal was to identify and fix a security oversight where project files were globally writable. Using `chmod`, write permissions were stripped from the "others" category to ensure data integrity. I determined `project_k.txt` must have the write access removed for other.

The following code demonstrates how I used Linux commands to do this:

```
drwx--x--- 2 researcher2 research_team 4096 Feb 22 18:46 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Feb 22 18:46 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 22 18:46 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_t.txt
researcher2@7a247624b384:~/projects$ chmod o-w project_k.txt
researcher2@7a247624b384:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Feb 22 18:46 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 22 18:46 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_t.txt
researcher2@7a247624b384:~/projects$
```

### Action Taken:

- **Initial Audit:** Executed `ls -l` to identify current file permissions. The file `project_k.txt` was found to have global write permissions (`-rw-rw-rw-`).
- **Permission Modification:** Used the `chmod` (change mode) command with the argument `o-w` to specifically remove **write** permissions from **others**.
- **Verification:** Re-ran `ls -l` to confirm the transition to `-rw-rw-r--`, ensuring that only the owner and the group retain editing rights.

## Change file permissions on a hidden file

The goal was to adjust the security profile of a hidden configuration file by restricting write access for the owner and group while granting read access to the group. This ensures the file remains readable for the team but prevents accidental modifications.

The following code demonstrates how I used Linux commands to change the permissions:

```
researcher2@7a247624b384:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Feb 22 18:46 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb 22 19:06 ..
-rw--w---- 1 researcher2 research_team  46 Feb 22 18:46 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Feb 22 18:46 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 22 18:46 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_t.txt
researcher2@7a247624b384:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@7a247624b384:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Feb 22 18:46 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb 22 19:06 ..
-r--r----- 1 researcher2 research_team  46 Feb 22 18:46 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Feb 22 18:46 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 22 18:46 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_t.txt
researcher2@7a247624b384:~/projects$
```

## Workflow

- **Audit:** Ran `ls -la` to view hidden files. The file `.project_x.txt` was initially set to `-rw--w----`.
- **Modification:** Used `chmod u-w,g-w,g+r` to perform a multi-class update: removing write access from both the owner and group, and adding read access for the group.
- **Verification:** Confirmed the change via `ls -la`, resulting in the updated permission string: `-r--r-----`.

## Change directory permissions

The goal was to isolate the `drafts` directory so that only the owner, `researcher2`, maintains access. By removing execute permissions from the group and others, the directory is effectively locked down, preventing anyone else from entering or listing its contents.

The following code demonstrates how I used Linux commands to change the permissions:

```
researcher2@7a247624b384:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Feb 22 18:46 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 22 18:46 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_t.txt
researcher2@7a247624b384:~/projects$ chmod g-x drafts
researcher2@7a247624b384:~/projects$ ls -l
total 20
drwx----- 2 researcher2 research_team 4096 Feb 22 18:46 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 22 18:46 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 22 18:46 project_t.txt
researcher2@7a247624b384:~/projects$
```

## Workflow

- **Audit:** Executed `ls -l` to check existing directory permissions. The `drafts` folder was initially set to `drwx--x---`, allowing group members to execute (enter) the directory.
- **Hardening:** Used `chmod g-x drafts` to strip the execute bit from the group. Since "others" already lacked permissions, this consolidated access solely to the owner.
- **Verification:** Confirmed the change via `ls -l`. The directory now reflects `drwx-----`, successfully restricting all access to `researcher2` alone.

## Summary

This project involved auditing and hardening file security within a Linux environment using the `chmod` utility. Access was restricted on the `project_k.txt` file by removing world-write permissions and on the hidden `.project_x.txt` file by shifting to a read-only group model. Additionally, the `drafts` directory was isolated by stripping group execute bits to ensure only the owner,

researcher2, could access its contents. Systematic verification was performed using `ls -la` to confirm that the principle of least privilege was strictly enforced across all files.